Course Notes

for (CSED490V) Foundations of Continuous Computation

Jungwoo Kim
Department of Mathematics
Pohang University of Science and Technology

Contents

Notation and Conventions **2**

1 Computable Reals **2**

2 Computable Functions **5**

3 Floating-Point Arithmetic **7**

Notation and Conventions

General notation and writing follow *(MATH000) Mathematical Language*.

Notation	Meaning
$\mathbb{R}_C$	Computable real numbers
$\mathcal{A}$	Algorithm (possibly using a real-number name)
$\gamma_{E,M}$	Floating-point representation map
$\mathbb{F}_{E,M}$, $\overline{\mathbb{F}}_{E,M}$	Finite and extended floating-point values
$\mathrm{rnd}_{E,M}$	Round to nearest, ties to even
$\oplus$, $\ominus$	Rounded addition and subtraction
$\otimes$, $\odot$	Rounded multiplication (primary, secondary)
$\oslash$, $\oslash$	Rounded division (primary, secondary)
fma	Fused multiply-add

1 Computable Reals

Definition 1.1. Borel Computability
A real number r is *Borel-computable* if there exists a computable function $q\colon \mathbb{N}_0 \to \mathbb{Q}$ such that

$$|r - q(n)| < 2^{-n} \qquad (n \in \mathbb{N}_0).$$

Definition 1.2. Turing Computability
A nonnegative real number r is *Turing-computable* if it has a binary expansion

$$r = a + \sum_{i=0}^{\infty} b_i 2^{-(i+1)}, \qquad a \in \mathbb{N}_0, \quad b_i \in \{0, 1\},$$

for which $i \mapsto b_i$ is computable. A negative real number is Turing-computable when its absolute value is Turing-computable. For dyadic rationals we may use the terminating binary expansion.

Theorem 1.3. Equivalence of Computability Definitions
For $r \in \mathbb{R}$, the following are equivalent:

(i) r is Borel-computable.

(ii) r is Turing-computable.

Proof. We may consider $r \geq 0$: changing the sign of a computable rational approximation is effective, and the negative case of binary computability is defined using the absolute value.

Suppose first that r is Turing-computable, with integer part a and computable bits b_i. Define

$$q(n) = a + \sum_{i=0}^{n} b_i 2^{-(i+1)} \in \mathbb{Q}.$$

The sum is computable and

$$0 \leq r - q(n) \leq \sum_{i=n+1}^{\infty} 2^{-(i+1)} = 2^{-(n+1)} < 2^{-n}.$$

Thus r is Borel-computable.

Conversely, suppose $q\colon \mathbb{N}_0 \to \mathbb{Q}$ computes r with $|q(m) - r| < 2^{-m}$. Fix the integer part $a = \lfloor r \rfloor$ as part of the existential description of a binary expansion. We distinguish two cases.

If r is dyadic, it admits an eventually zero binary expansion. Its finite sequence of nonzero digits can be built into an algorithm, and hence r is Turing-computable.

Now assume r is not dyadic. Fix $n \in \mathbb{N}_0$. The non-dyadic number r lies strictly between two consecutive grid points of mesh $2^{-(n+1)}$. Search over $m \in \mathbb{N}_0$ until the rational interval

$$I_m = (q(m) - 2^{-m},\, q(m) + 2^{-m})$$

is contained strictly in a single grid cell

$$\left(\frac{k}{2^{n+1}}, \frac{k+1}{2^{n+1}}\right), \qquad k \in \mathbb{N}_0.$$

Containment is decidable by rational arithmetic: for a given m there is at most one candidate k, which can be found by integer division. The search terminates, since r has positive distance from both endpoints of its grid cell and $q(m) \to r$ with an effective error bound. Moreover $r \in I_m$, so the returned k satisfies $k = \lfloor 2^{n+1} r \rfloor$. The nth binary digit of r is therefore

$$b_n = k \bmod 2.$$

This procedure computes each bit, completing the proof. □

Remark. The two definitions characterize the same set of real numbers, but their representations do not admit symmetric *uniform* conversions. Given a computable binary expansion one can effectively obtain rational approximations. In general there is no algorithm that takes an arbitrary rational-approximation procedure and uniformly produces its binary digits: the latter operation must resolve boundary cases at dyadic rationals. This does not contradict the existence of a computable binary expansion for each Borel-computable real.

Definition 1.4. Computable Real Numbers
The set of *computable real numbers* is

$$\mathbb{R}_C \coloneqq \{r \in \mathbb{R} \mid r \text{ is computable}\}.$$

By the preceding theorem, either notion of computability can be used.

Proposition 1.5. Closure under Arithmetic
The set $\mathbb{R}_C$ is a subfield of $\mathbb{R}$: if $x, y \in \mathbb{R}_C$, then

$$x + y, \quad x - y, \quad xy \in \mathbb{R}_C, \qquad \frac{x}{y} \in \mathbb{R}_C \quad \textit{when } y \neq 0.$$

Proof. Take computable rational approximations to x and y with arbitrarily small certified error. Addition and subtraction preserve effective error bounds by the triangle inequality. For multiplication, first obtain rational upper bounds on $|x|$ and $|y|$, then use

$$|xy - uv| \leq |x|\,|y - v| + |v|\,|x - u|$$

to choose sufficiently accurate rational u, v. Finally, if $y \neq 0$, search for an approximation v whose certified error is smaller than $|v|$; this eventually succeeds and supplies an effective positive lower bound on $|y|$. Rational approximations to $1/y$, and hence x/y, follow with computable error bounds. All four constructions use only rational arithmetic. □

Proposition 1.6. Real Algebraic Closure
If $p \in \mathbb{R}_C[X]$ is nonzero, then every real root of p lies in $\mathbb{R}_C$. In particular, $\mathbb{R}_C$ is a real-closed field.

Proof. Induct on $\deg p$. Constant polynomials have no roots. Let r be a real root of p of positive degree. If $p'(r) = 0$, then r is a root of the nonzero polynomial $p' \in \mathbb{R}_C[X]$, so the induction hypothesis applies. Otherwise r is simple. Choose rationals $a < r < b$ so that r is the only root in $[a, b]$ and $p(a)p(b) < 0$. Since $p(a)$ and $p(b)$ are nonzero computable reals, their signs can be determined effectively. If $r \in \mathbb{Q}$, it is already computable. Otherwise each rational midpoint in a bisection of $[a, b]$ is not a root, and the sign of its polynomial value can likewise be computed. Bisection therefore produces rational approximations to r with any prescribed error. Hence $r \in \mathbb{R}_C$. □

Remark. This is closure under *real* algebraic roots, not algebraic closure: $\mathbb{R}_C$ is not algebraically closed over $\mathbb{C}$.

Proposition 1.7. Computability of π and e
The real numbers π and e are computable.

Proof. The alternating arctangent series gives

$$\frac{\pi}{4} = \arctan 1 = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}.$$

For $m \in \mathbb{N}_0$ put

$$p_m = 4 \sum_{k=0}^{m} \frac{(-1)^k}{2k+1} \in \mathbb{Q}.$$

The alternating-series estimate yields

$$|\pi - p_m| \leq \frac{4}{2m+3}.$$

Choosing, for instance, $m = 2^{n+2}$ gives $|\pi - p_m| < 2^{-n}$ for every $n \in \mathbb{N}_0$. The finite sum p_m is computable from n.

Similarly, let $e_m = \sum_{k=0}^{m} 1/k!$. For $m \geq 1$,

$$0 < e - e_m = \sum_{k=m+1}^{\infty} \frac{1}{k!} \leq \frac{2}{(m+1)!}.$$

Taking $m = n + 3$ makes the error strictly smaller than 2^{-n}. Thus e is computable as well. □

Proposition 1.8. Countability of the Computable Reals
The set $\mathbb{R}_C$ is countably infinite. Consequently, there exist noncomputable real numbers.

Proof. There are countably many finite descriptions of algorithms, and every computable real has at least one such description. Hence $\mathbb{R}_C$ is at most countable. On the other hand, every rational number is computable via its constant approximation sequence, so $\mathbb{Q} \subseteq \mathbb{R}_C$ and $\mathbb{R}_C$ is infinite. Since $\mathbb{R}$ is uncountable, $\mathbb{R} \setminus \mathbb{R}_C \neq \varnothing$. □

Fact 1.9. Hilbert's Tenth Problem
There is no algorithm that decides, for every multivariate polynomial with integer coefficients, whether it has an integer zero. In particular, the set of integer polynomials admitting an integer solution is not decidable.

Proposition 1.10. A Noncomputable Real
Let P be the set of all integer-coefficient polynomials in finitely many variables, and let $\phi\colon P \to \mathbb{N}_0$ be a computable bijective encoding. Write P_{root} for the polynomials having an integer zero. Then

$$r := \sum_{p \in P} 2^{-(\phi(p)+1)}\, \mathbb{1}_{P_{\text{root}}}(p)$$

is a noncomputable real in $[0,1]$.

Proof. The binary digit at position $\phi(p)$ is 1 precisely when $p \in P_{\text{root}}$. Both this set and its complement are infinite; for example, among polynomials of one variable there are infinitely many with an integer zero and infinitely many without one. Thus the binary sequence has infinitely many zeros and ones, so the expansion is neither eventually zero nor eventually one and r is not dyadic.

If r were computable, the equivalence theorem would give a computable binary expansion. Because r is not dyadic, its binary expansion is unique; its digit at position $\phi(p)$ could be computed from p. This would decide whether p has an integer zero, contradicting Hilbert's Tenth Problem. □

2 Computable Functions

Definition 2.1. Computable Function
Let $D \subseteq \mathbb{R}$. A function $f\colon D \to \mathbb{R}$ is *computable* if there exists an oracle algorithm $\mathcal{A}$ with the following property. For every $x \in D$, every function $\alpha\colon \mathbb{N}_0 \to \mathbb{Q}$ satisfying

$$|\alpha(i) - x| < 2^{-i} \qquad (i \in \mathbb{N}_0),$$

and every precision $n \in \mathbb{N}_0$, the algorithm $\mathcal{A}$, given access to α and the input n, halts and returns a rational number $\mathcal{A}^{\alpha}(n)$ such that

$$|\mathcal{A}^{\alpha}(n) - f(x)| < 2^{-n}.$$

The algorithm may query finitely many values $\alpha(i)$ during a computation. The definition extends componentwise to functions between subsets of $\mathbb{R}^k$ and $\mathbb{R}^m$.

Remark. The input name α need not itself be computable: the algorithm must work for *every* sufficiently accurate rational name of x, including names of noncomputable reals. What must be computable is the procedure transforming such a name into approximations to $f(x)$.

In particular, every computable $f\colon D \to \mathbb{R}$ satisfies $f(D \cap \mathbb{R}_C) \subseteq \mathbb{R}_C$.

Example 2.2. Basic Computable Functions
For $r \in \mathbb{R}_C$, both $x \mapsto x + r$ and $x \mapsto rx$ are computable. Other examples are $|x|$, e^x, $\sin x$, $\sqrt{x}$ (for $x \geq 0$), and $\log x$ (for $x > 0$).

In several variables, $+$, $-$, multiplication, min, and max are computable; division is computable when the divisor is nonzero.

Theorem 2.3. Closure under Composition
Let $f\colon D \to E$ and $g\colon E \to \mathbb{R}^m$ be computable, where $D \subseteq \mathbb{R}^k$ and $E \subseteq \mathbb{R}^\ell$. Then $g \circ f$ is computable.

Proof. For a name of $\mathbf{x} \in D$, the algorithm for f produces $\mathbf{u}_j \in \mathbb{Q}^\ell$ on demand such that

$$\|\mathbf{u}_j - f(\mathbf{x})\|_\infty < 2^{-(j+2)}.$$

Thus $j \mapsto \mathbf{u}_j$ is a name of $f(\mathbf{x})$. Supply this name to the algorithm for g: every query is answered by finitely many computations of f, so the procedure halts and computes $g(f(\mathbf{x}))$. □

Theorem 2.4. Computability Implies Continuity
Every computable function $f\colon D \to \mathbb{R}^m$, with $D \subseteq \mathbb{R}^k$, is continuous on D (with the relative topology).

Proof. Fix $\mathbf{x} \in D$ and $\varepsilon > 0$; choose $n \in \mathbb{N}_0$ with $2^{1-n} < \varepsilon$. Choose a rational name $\boldsymbol{\alpha}\colon \mathbb{N}_0 \to \mathbb{Q}^k$ of $\mathbf{x}$ with slack:

$$\|\boldsymbol{\alpha}(i) - \mathbf{x}\|_\infty < 2^{-(i+2)} \qquad (i \in \mathbb{N}_0).$$

At precision n, the algorithm queries a finite set of indices I. For some $\delta > 0$, every $\mathbf{y} \in D$ with $\|\mathbf{x} - \mathbf{y}\|_\infty < \delta$ also admits a name agreeing with $\boldsymbol{\alpha}$ on I. The algorithm therefore returns the same $\mathbf{q} \in \mathbb{Q}^m$ on both names, whence

$$\|f(\mathbf{x}) - f(\mathbf{y})\|_\infty \le \|f(\mathbf{x}) - \mathbf{q}\|_\infty + \|\mathbf{q} - f(\mathbf{y})\|_\infty < 2^{1-n} < \varepsilon.$$

Thus f is continuous at $\mathbf{x}$. □

Remark. Continuity is necessary, not sufficient, for computability: if $c \in \mathbb{R} \setminus \mathbb{R}_C$, the constant function $x \mapsto c$ is continuous but not computable. In particular, a function with a discontinuity at a point of its domain cannot be computable there.

Example 2.5. Discontinuous Functions and Their Domains
The step function $x \mapsto \mathbb{1}_{\{x>0\}}$ is not computable on $\mathbb{R}$, since it is discontinuous at 0. Its restriction to $\mathbb{R} \setminus \{0\}$ is computable: approximate the nonzero input until its sign can be certified, and return the corresponding bit. The same principle applies to strict comparison when $x \neq y$ (see below).

Theorem 2.6. Computability of the Exponential Function
The function $\exp\colon \mathbb{R} \to \mathbb{R}$ *is computable.*

Proof. First, let $x \in [-\frac{1}{2}, \frac{1}{2}]$, with rational name α, and fix $n \in \mathbb{N}_0$. Put $m = d = n + 4$ and

$$t = \alpha(m) \in \mathbb{Q}, \qquad P_d(t) = \sum_{j=0}^{d} \frac{t^j}{j!} \in \mathbb{Q}.$$

Since $|t - x| < 2^{-m}$, we have $|t| < 1$. The mean value theorem and the Taylor remainder give

$$|e^x - e^t| \le 3 \cdot 2^{-m} < 2^{-(n+1)},$$
$$|e^t - P_d(t)| \le \frac{2}{(d+1)!} < 2^{-(n+1)}.$$

Hence $|e^x - P_d(t)| < 2^{-n}$, proving computability on the interval.

For arbitrary $x \in \mathbb{R}$, choose $k \in \mathbb{N}_0$ effectively from a coarse rational approximation so that $|x/2^k| \le \frac{1}{2}$. Compute $u = e^{x/2^k}$ to sufficient precision by the preceding procedure, then approximate $u^{2^k} = e^x$ using rational multiplication. Writing $N = 2^k$, choose a rational approximation v to u with $|u - v| < 1$. Since $u \in (0, 2)$, the estimate

$$|u^N - v^N| \le N 3^{N-1} |u - v|$$

shows that a sufficiently accurate v yields any prescribed error. □

Theorem 2.7. Noncomputability of Strict Comparison
The strict-comparison function

$$L\colon \mathbb{R}^2 \to \{0, 1\}, \qquad L(x, y) = \begin{cases} 1, & x < y, \\ 0, & x \ge y, \end{cases}$$

is not computable.

Proof. Suppose an algorithm computes L. Because its values lie in $\{0,1\}$, requesting an approximation with error below $1/4$ and testing whether the result exceeds $1/2$ yields an algorithm returning the exact bit. Give this decision algorithm the two names

$$\alpha(i) = 0, \qquad \beta(i) = 2^{-(i+1)} \qquad (i \in \mathbb{N}_0),$$

both of which represent the real number 0. Hence the algorithm must halt and output 0. During this run, only finitely many indices of β are queried; let $N \in \mathbb{N}_0$ exceed or equal all of them. Define another rational name by

$$\beta'(i) = \begin{cases} 2^{-(i+1)}, & i \le N, \\ 2^{-(N+1)}, & i > N. \end{cases}$$

For $i \le N$, its distance from $2^{-(N+1)}$ is strictly smaller than 2^{-i}, and for $i > N$ the distance is zero. Thus β' is a valid name of $y' = 2^{-(N+1)} > 0$. The algorithm cannot distinguish (α, β') from (α, β) during its finite computation; it still returns 0. But $L(0, y') = 1$, a contradiction. □

Proposition 2.8. Comparison of Unequal Real Numbers
The restriction

$$L\colon \{(x,y) \in \mathbb{R}^2 \mid x \neq y\} \to \{0,1\}$$

of strict comparison is computable.

Proof. Given names α, β of x, y, inspect successively the rational intervals

$$I_n = (\alpha(n) - 2^{-n},\, \alpha(n) + 2^{-n}), \qquad J_n = (\beta(n) - 2^{-n},\, \beta(n) + 2^{-n}).$$

If $\sup I_n < \inf J_n$, output 1; if $\sup J_n < \inf I_n$, output 0. These comparisons are rational and hence decidable. Since $x \neq y$ and the intervals shrink to their respective points, eventually one of the tests succeeds. The sign of $x - y$ is then certified, so the answer is correct. □

Remark. Without the promise $x \neq y$, equality cannot be decided from a finite amount of rational approximation data. Under that promise, the separating intervals above provide a terminating comparison procedure.

3 Floating-Point Arithmetic

Definition 3.1. Floating-Point Representation
Fix $E \ge 2$ and $M \ge 1$. A floating-point bit string consists of a sign bit s, E exponent bits, and M fraction bits. Let

$$k = (e_1 \cdots e_E)_2, \qquad t = (0.m_1 \cdots m_M)_2 = \sum_{j=1}^{M} m_j 2^{-j}, \qquad b_E = 2^{E-1} - 1.$$

The interpretation map $\gamma_{E,M}\colon \{0,1\}^{1+E+M} \to \mathbb{R} \cup \{-\infty, +\infty, \mathrm{NaN}\}$ is defined by

$$\gamma_{E,M}(s, e_1, \ldots, e_E, m_1, \ldots, m_M) = \begin{cases} (-1)^s 2^{k-b_E}(1+t), & 1 \le k \le 2^E - 2, \\ (-1)^s 2^{1-b_E} t, & k = 0, \\ (-1)^s \infty, & k = 2^E - 1,\ t = 0, \\ \mathrm{NaN}, & k = 2^E - 1,\ t \neq 0. \end{cases}$$

The first two cases give *normal* and *subnormal* values; the remaining cases represent infinities and *not a number* (NaN). Write

$$\overline{\mathbb{F}}_{E,M} = \operatorname{im} \gamma_{E,M}, \qquad \mathbb{F}_{E,M} = \overline{\mathbb{F}}_{E,M} \setminus \{-\infty, +\infty, \mathrm{NaN}\}.$$

Thus $\mathbb{F}_{E,M} \subseteq \mathbb{R}$ is the set of finite floating-point values.

Remark. The two signed-zero bit strings both represent $0 \in \mathbb{F}_{E,M}$, while several distinct bit strings represent NaN. The map $\gamma_{E,M}$ identifies these representations as values. In the usual binary32 and binary64 formats, $(E, M) = (8, 23)$ and $(11, 52)$, respectively.

Definition 3.2. Rounding
Let

$$F_{\max} = (2 - 2^{-M})2^{b_E}, \qquad K_{E,M} = (2 - 2^{-M-1})2^{b_E}.$$

The *round-to-nearest, ties-to-even* map $\mathrm{rnd}_{E,M} \colon \mathbb{R} \to \overline{\mathbb{F}}_{E,M}$ is defined by

$$\mathrm{rnd}_{E,M}(x) = \begin{cases} +\infty, & x \geq K_{E,M}, \\ -\infty, & x \leq -K_{E,M}, \\ \operatorname*{arg\,min}_{y \in \mathbb{F}_{E,M}} |x - y|, & |x| < K_{E,M}. \end{cases}$$

If two finite values minimize the distance, choose the one whose least significant stored significand bit is even. In particular, halfway cases near 0 round to 0; the chosen value always exists and is unique.

Proposition 3.3. Monotonicity of Rounding
For $x, y \in \mathbb{R}$,

$$x \leq y \quad \Longrightarrow \quad \mathrm{rnd}_{E,M}(x) \leq \mathrm{rnd}_{E,M}(y),$$

with the usual order on $\mathbb{R} \cup \{-\infty, +\infty\}$.

Proof. Overflow regions preserve order, so consider finite rounded values $u = \mathrm{rnd}_{E,M}(x)$ and $v = \mathrm{rnd}_{E,M}(y)$. If $u > v$, optimality of nearest rounding gives

$$|x - u| \leq |x - v| \quad \Longrightarrow \quad x \geq \frac{u+v}{2}, \qquad |y - v| \leq |y - u| \quad \Longrightarrow \quad y \leq \frac{u+v}{2}.$$

Together with $x \leq y$, this forces $x = y = (u+v)/2$, contrary to the uniqueness of the ties-to-even choice. Hence $u \leq v$. □

Definition 3.4. Floating-Point Operations
For $a, b, c \in \mathbb{F}_{E,M}$, define rounded addition, subtraction, and multiplication by

$$\begin{aligned} a \oplus b &= \mathrm{rnd}_{E,M}(a + b), \quad & a \ominus b &= \mathrm{rnd}_{E,M}(a - b), \\ a \otimes b &= \mathrm{rnd}_{E,M}(ab), & a \oslash b &= \mathrm{rnd}_{E,M}(a/b) \quad (b \neq 0). \end{aligned}$$

The secondary notations are $\odot$ for multiplication and $\oslash$ for division. *Fused multiply-add* performs only one rounding:

$$\mathrm{fma}(a, b, c) = \mathrm{rnd}_{E,M}(ab + c).$$

These operations have values in $\overline{\mathbb{F}}_{E,M}$; operations involving infinities, NaN, or signed-zero bit patterns are outside the present finite-input definitions.

Remark. Rounded arithmetic is not a field. Although $\oplus$ and $\otimes$ are commutative on finite inputs, they need not be associative, and multiplication need not distribute over addition. For example, in the format $(E, M) = (3, 2)$, ties to even gives

$$(1 \oplus \tfrac{1}{8}) \oplus (-1) = 0 \neq \tfrac{1}{8} = 1 \oplus (\tfrac{1}{8} \oplus (-1)).$$

Also, finite values are not closed under these rounded operations when an overflow produces $\pm\infty$.